Lab Work #3 : The Schemas War ⚔️

IMDb Non-Commercial Dataset
https://developer.imdb.com/non-commercial-datasets/

Course: RDMS for Production

Duration: 3 hours

Groups: 2–3 students

Author: Samuel Rohaut

Context

So far, you've imported raw data into flat staging tables. That's fine for a data dump, but it's terrible for production. In the real world, the same data gets structured very differently depending on how it's used:

- **OLTP** (Online Transaction Processing):
  - Serve a website, an API and/or a mobile app.
  - Fast point lookups, joins, searches.
  - Lots of small reads, some writes.
  - Think: `SELECT * FROM titles WHERE id = ?`
- **OLAP** (Online Analytical Processing):
  - Answer business questions.
  - Scans over millions of rows, aggregations, GROUP BY everywhere.
  - Think: `SELECT decade, genre, AVG(rating) FROM ... GROUP BY ...`

In this lab, you'll design **two separate schemas** from the same staging data — one for each purpose.

Prerequisites

All staging tables from Lab 1 must be loaded and working, using the tables' names defined in Lab #1.

Part 1 — OLTP: Build IMDb.com's Clone 🌐

You're building the backend for a website like IMDb. Users visit pages, search for titles, browse actors. Every query must return in **under 50ms** for a good user experience.

1.1 — Design the Normalized Schema

Design a properly normalized (3NF) schema. Here's what the website needs to serve:

Title detail pages: Display a title with its genres, rating, cast, crew, and episode info

Person pages: Display a person with their filmography, roles, and known-for titles

Search: Find titles by name (prefix search and full-text)

Browse: List titles by genre, by year, by type

Top lists: Top rated movies, most popular this year, etc.

Required Tables (minimum)

Design at least these tables (you may add more):

Table	Purpose
`titles`	One row per title (movies, series, episodes)
`genres`	Genre lookup table
`title_genres`	Many-to-many: title ↔ genre
`people`	One row per person
`title_cast`	Who acted/worked in what (from principals)
`title_crew`	Directors and writers per title
`ratings`	Ratings (or merge into titles — your choice)
`episodes`	Episode ↔ series relationship

For each table, specify:

- Column names and **appropriate types** (learned from Lab 1!)
- Primary keys & Foreign keys
- NOT NULL constraints and/or any CHECK constraints that make sense

Deliverable

Write the complete `CREATE TABLE` statements for your OLTP schema.

1.2 — Migrate from Staging

Write SQL queries to populate your normalized tables from the staging tables.

Handle:

- **Genre normalization:** Extract unique genres into a `genres` table, then populate `title_genres`
- **Crew explosion:** The `directors` and `writers` fields are comma-separated nconst lists, unnest them into proper rows
- **NULL handling:** Remember that `\N` in the staging data became SQL NULL after import
- **Type casting:** Any type conversions needed

Example: Genre Normalization

```sql
-- Create the genre lookup
INSERT INTO genres (name)
SELECT DISTINCT genre
FROM stg_title_basics, LATERAL unnest(string_to_array(genres, ',')) AS genre
WHERE genre IS NOT NULL AND genre != ''
ORDER BY genre;

-- Populate the bridge table
INSERT INTO title_genres (title_id, genre_id)
SELECT tb.tconst, g.id
FROM stg_title_basics tb,
     LATERAL unnest(string_to_array(tb.genres, ',')) AS genre_name
JOIN genres g ON g.name = genre_name;
```

Deliverable

Write the complete migration script (all `INSERT INTO ... SELECT` statements).

1.3 — Denormalized Display Tables

Normalization is great for consistency, but joining 5 tables on every page load is expensive. Create **denormalized materialized views or tables** for the two most common page types:

`title_display` — Everything needed for a title page

```
CREATE TABLE title_display (
    tconst          VARCHAR(12) PRIMARY KEY,
    primary_title   TEXT NOT NULL,
    original_title  TEXT,
    title_type      VARCHAR(20),
    start_year      INTEGER,
    end_year        INTEGER,
    runtime_minutes INTEGER,
    is_adult        BOOLEAN,
    genres          TEXT[],                -- array of genre names
    average_rating  DECIMAL(3,1),
    num_votes       INTEGER,
    top_cast        JSONB,                 -- [{name, role, ordering}, ...]
    directors       TEXT[],                -- array of director names
    writers         TEXT[]                 -- array of writer names
);
```

`person_display` — Everything needed for a person page

```
CREATE TABLE person_display (
    nconst              VARCHAR(12) PRIMARY KEY,
    primary_name        TEXT NOT NULL,
    birth_year          INTEGER,
    death_year          INTEGER,
    primary_professions TEXT[],
    filmography         JSONB, -- [{tconst, title, year, role, rating}, ...]
    known_for           JSONB  -- [{tconst, title, year, rating}, ...]
);
```

Deliverable

1. `CREATE TABLE` statements for both display tables
2. `INSERT INTO ... SELECT` queries to populate them from your normalized tables
3. **Triggers** (or a documented refresh strategy) to keep them in sync when the underlying data changes

Example Trigger (sync rating updates):

```
CREATE OR REPLACE FUNCTION sync_title_display_rating()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE title_display
    SET average_rating = NEW.average_rating,
        num_votes = NEW.num_votes
    WHERE tconst = NEW.tconst;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_rating_update
AFTER INSERT OR UPDATE ON ratings
FOR EACH ROW EXECUTE FUNCTION sync_title_display_rating();
```

Write similar triggers for at least:

- Rating changes → update `title_display`
- Cast changes → update `title_display.top_cast`
- Title changes → update both `title_display` and relevant `person_display.filmography`

1.4 — OLTP Indexes

Create indexes optimized for website access patterns:

```
-- Point lookups (already covered by PKs)
-- Search by title
CREATE INDEX idx_titles_search
  ON titles USING gin (primary_title gin_trgm_ops);
-- Or prefix search:
CREATE INDEX idx_titles_prefix
  ON titles (primary_title varchar_pattern_ops);

-- Browse by genre
CREATE INDEX idx_title_genres_genre
  ON title_genres (genre_id);

-- Browse by year + type
CREATE INDEX idx_titles_type_year
  ON titles (title_type, start_year DESC);

-- Top rated
CREATE INDEX idx_ratings_top
  ON ratings (average_rating DESC) WHERE num_votes > 10000;
```

```sql
-- Episode lookup
CREATE INDEX idx_episodes_parent
  ON episodes (parent_tconst, season_number, episode_number);
```

Adapt these to your schema. Add any others you think are needed.

1.5 — Benchmark OLTP Queries

Run these 5 queries with `EXPLAIN ANALYZE` and record the execution time:

OQ1 — Title Page

```sql
-- Fetch the complete title page for "The Shawshank Redemption"
SELECT * FROM title_display WHERE tconst = 'tt0111161';
```

OQ2 — Search

```sql
-- Search for titles starting with "The Dark"
SELECT tconst, primary_title, title_type, start_year, average_rating,
num_votes
FROM title_display
WHERE primary_title LIKE 'The Dark%'
ORDER BY num_votes DESC NULLS LAST
LIMIT 20;
```

OQ3 — Person Page

```sql
-- Fetch Christopher Nolan's page
SELECT * FROM person_display WHERE nconst = 'nm0634240';
```

OQ4 — Top Movies 2023

```sql
-- Top 20 movies of 2023
SELECT tconst, primary_title, average_rating, num_votes
FROM title_display
WHERE title_type = 'movie'
  AND start_year = 2023
  AND num_votes >= 10000
ORDER BY average_rating DESC
LIMIT 20;
```

OQ5 — Browse Horror

```sql
-- 10 most recent horror movies with ratings
SELECT td.tconst, td.primary_title, td.start_year, td.average_rating
FROM title_display td
WHERE 'Horror' = ANY(td.genres)
  AND td.title_type = 'movie'
  AND td.average_rating IS NOT NULL
ORDER BY td.start_year DESC, td.average_rating DESC
LIMIT 10;
```

Target: Each query should complete in **under 50ms**. If not, revisit your indexes.

Part 2 — OLAP: IMDb Pro Analytics 📊

"All the stats you always wanted to know about movies, series, and people… but you were afraid to ask for, because… it was dead slow 👻"

You're building the analytics backend for IMDb Pro. Analysts run complex queries spanning millions of rows. Individual query time is less critical (seconds are OK), but the schema must be optimized for aggregations, GROUP BY, and large scans.

2.1 — Design the Star Schema

Design a classic star/snowflake schema:

Fact Table

```sql
CREATE TABLE fact_title (
    title_key       SERIAL PRIMARY KEY,
    tconst          VARCHAR(12) UNIQUE NOT NULL,
    title_type_key  INTEGER REFERENCES dim_title_type(title_type_key),
    time_key        INTEGER REFERENCES dim_time(time_key),
    -- Measures
    runtime_minutes INTEGER,
    average_rating  DECIMAL(3,1),
    num_votes       INTEGER,
    is_adult        BOOLEAN,
    -- Pre-aggregated
    cast_count      INTEGER,    -- number of cast members
    genre_count     INTEGER     -- number of genres
);
```

Dimension Tables

```sql
CREATE TABLE dim_time (
    time_key    SERIAL PRIMARY KEY,
    year        INTEGER,
    decade      INTEGER,        -- 1990, 2000, 2010...
    century     INTEGER,        -- 19, 20, 21
    decade_label VARCHAR(10)    -- '1990s', '2000s'...
);

CREATE TABLE dim_genre (
    genre_key   SERIAL PRIMARY KEY,
    genre_name  VARCHAR(50) UNIQUE NOT NULL
);
```

```sql
CREATE TABLE dim_title_type (
    title_type_key  SERIAL PRIMARY KEY,
    title_type      VARCHAR(20) UNIQUE NOT NULL
);

CREATE TABLE dim_person (
    person_key          SERIAL PRIMARY KEY,
    nconst              VARCHAR(12) UNIQUE NOT NULL,
    primary_name        TEXT,
    birth_year          INTEGER,
    birth_decade        INTEGER,
    death_year          INTEGER,
    is_alive            BOOLEAN,
    primary_profession  TEXT        -- keep as-is for filtering
);
```

Bridge Tables (many-to-many)

```sql
CREATE TABLE bridge_title_genre (
    title_key   INTEGER REFERENCES fact_title(title_key),
    genre_key   INTEGER REFERENCES dim_genre(genre_key),
    PRIMARY KEY (title_key, genre_key)
);

CREATE TABLE bridge_title_person (
    title_key   INTEGER REFERENCES fact_title(title_key),
    person_key  INTEGER REFERENCES dim_person(person_key),
    role_category VARCHAR(50),   -- 'actor', 'director', 'writer', etc.
    ordering    INTEGER,
    PRIMARY KEY (title_key, person_key, role_category)
);
```

Deliverable

Write the complete star schema DDL. You may adapt the above or design your own — just justify your choices.

2.2 — ETL: Populate the Star Schema

Write the ETL (Extract-Transform-Load) queries to populate your star schema from the staging tables.

Order matters:

1. Dimension tables first (they have no dependencies)
2. Fact table second (references dimensions)
3. Bridge tables last (reference both facts and dimensions)
4.

Example: Populate dim_time

```
INSERT INTO dim_time (year, decade, century, decade_label)
SELECT DISTINCT
    startYear,
    (startYear / 10) * 10,
    startYear / 100 + 1,
    ((startYear / 10) * 10)::TEXT || 's'
FROM stg_title_basics
WHERE startYear IS NOT NULL
ORDER BY startYear;
```

Deliverable

Write the complete ETL script. Time it and report how long each step takes.

2.3 — OLAP Indexes

Different access patterns require different indexes. For OLAP, think: - Range scans on dimensions (years, decades) - Join performance on bridge tables - Covering indexes for common aggregations

```
-- Time dimension for range queries
CREATE INDEX idx_dim_time_decade ON dim_time (decade);
CREATE INDEX idx_dim_time_year ON dim_time (year);

-- Fact table joins and filters
CREATE INDEX idx_fact_title_time ON fact_title (time_key);
CREATE INDEX idx_fact_title_type ON fact_title (title_type_key);
CREATE INDEX idx_fact_title_rating ON fact_title (average_rating) WHERE
average_rating IS NOT NULL;
```

```sql
-- Bridge tables (already have PKs, but might need reverse lookups)
CREATE INDEX idx_bridge_genre_reverse ON bridge_title_genre (genre_key,
title_key);
CREATE INDEX idx_bridge_person_reverse ON bridge_title_person (person_key,
title_key);
CREATE INDEX idx_bridge_person_role ON bridge_title_person (role_category,
person_key);
```

Adapt to your schema.

2.4 — Benchmark OLAP Queries

Run these 5 analytical queries with `EXPLAIN ANALYZE`:

AQ1 — Genre Popularity by Decade

```sql
-- How has genre popularity evolved over the decades?
SELECT dt.decade_label, dg.genre_name, COUNT(*) AS title_count
FROM fact_title ft
JOIN dim_time dt ON ft.time_key = dt.time_key
JOIN bridge_title_genre btg ON ft.title_key = btg.title_key
JOIN dim_genre dg ON btg.genre_key = dg.genre_key
WHERE dt.decade >= 1950
GROUP BY dt.decade_label, dt.decade, dg.genre_name
ORDER BY dt.decade, title_count DESC;
```

AQ2 — Most Prolific Actors per Decade

```sql
-- Top 5 actors by number of titles per decade
SELECT dt.decade_label, dp.primary_name, COUNT(*) AS title_count
FROM fact_title ft
JOIN dim_time dt ON ft.time_key = dt.time_key
JOIN bridge_title_person btp ON ft.title_key = btp.title_key
JOIN dim_person dp ON btp.person_key = dp.person_key
WHERE btp.role_category IN ('actor', 'actress')
  AND dt.decade >= 1950
GROUP BY dt.decade_label, dt.decade, dp.primary_name
ORDER BY dt.decade, title_count DESC;
-- (filter to top 5 per decade using window function in a CTE)
```

AQ3 — Rating Distribution

```sql
-- Distribution of ratings (histogram)
SELECT
    FLOOR(average_rating) AS rating_bucket,
    COUNT(*) AS count,
    ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 2) AS percentage
FROM fact_title
WHERE average_rating IS NOT NULL
GROUP BY rating_bucket
ORDER BY rating_bucket;
```

AQ4 — Runtime Trend by Decade

```sql
-- Average movie runtime per decade
SELECT dt.decade_label,
       ROUND(AVG(ft.runtime_minutes), 1) AS avg_runtime,
       COUNT(*) AS movie_count
FROM fact_title ft
JOIN dim_time dt ON ft.time_key = dt.time_key
JOIN dim_title_type dtt ON ft.title_type_key = dtt.title_type_key
WHERE dtt.title_type = 'movie'
  AND ft.runtime_minutes IS NOT NULL
  AND ft.runtime_minutes BETWEEN 30 AND 300
  AND dt.decade >= 1920
GROUP BY dt.decade_label, dt.decade
ORDER BY dt.decade;
```

AQ5 — Director Career Trajectory

```sql
-- For directors with 10+ movies: compare first-half vs second-half average
rating
WITH director_movies AS (
    SELECT dp.person_key, dp.primary_name,
           ft.average_rating, ft.num_votes,
           dt.year,
           ROW_NUMBER() OVER (PARTITION BY dp.person_key ORDER BY dt.year) AS
movie_num,
           COUNT(*) OVER (PARTITION BY dp.person_key) AS total_movies
    FROM bridge_title_person btp
    JOIN dim_person dp ON btp.person_key = dp.person_key
    JOIN fact_title ft ON btp.title_key = ft.title_key
    JOIN dim_time dt ON ft.time_key = dt.time_key
    JOIN dim_title_type dtt ON ft.title_type_key = dtt.title_type_key
    WHERE btp.role_category = 'director'
      AND dtt.title_type = 'movie'
      AND ft.average_rating IS NOT NULL
)
```

```sql
SELECT primary_name,
       total_movies,
       ROUND(AVG(CASE WHEN movie_num <= total_movies / 2 THEN average_rating
END), 2) AS first_half_avg,
       ROUND(AVG(CASE WHEN movie_num > total_movies / 2 THEN average_rating
END), 2) AS second_half_avg,
       ROUND(AVG(CASE WHEN movie_num > total_movies / 2 THEN average_rating
END) -
             AVG(CASE WHEN movie_num <= total_movies / 2 THEN average_rating
END), 2) AS improvement
FROM director_movies
WHERE total_movies >= 10
GROUP BY person_key, primary_name, total_movies
ORDER BY improvement DESC
LIMIT 20;
```

Part 3 — Compare & Reflect

In your report, answer these questions:

1. **Schema size:** How much disk space does each schema use? (Use `pg_total_relation_size`)
2. **ETL time:** How long did it take to populate each schema?
3. **Query performance:** Which schema is faster for which type of query?
4. **Maintenance:** Which schema is easier to update when new data arrives?
5. **Trade-offs:** If you had to pick ONE schema for a system that does both website serving and analytics, what compromises would you make?

```sql
-- Measure schema sizes
SELECT
    schemaname,
    tablename,
    pg_size_pretty(pg_total_relation_size(schemaname || '.' || tablename)) AS
total_size
FROM pg_tables
WHERE schemaname = 'public'
ORDER BY pg_total_relation_size(schemaname || '.' || tablename) DESC;
```

Deliverables

1. **OLTP Schema:** Complete DDL, migration script, denormalized tables, triggers, indexes, benchmark results
2. **OLAP Schema:** Complete DDL, ETL script, indexes, benchmark results
3. **Comparison report** answering the questions in Part 3

⚠️ **Keep your SQL scripts clean and well-organized — you'll need them for Lab 4!**

Two schemas diverged in a database, and you — you took the one that fit the workload, and that has made all the difference. 🌿